DICTIONARY

GAME PROGRAMMING COURSE — C# BASICS

Paiza 辞書

手が止まったら引く、C# の書き方まとめ

次のページの「もくじ」から探そう

もくじ

困ったときに引くページ

A **基本の形**
Paiza の書き始め p.3

B **入力**
1 行を受け取る・スペース区切りを受け取る・複数行を受け取る p.4〜6

C **出力**
出力する p.7

D **計算**
計算でよく使う書き方 p.8

E **if 文**
if 文の書き方・条件の書き方 p.9〜10

F **for 文**
for 文の書き方 ①・for 文の書き方 ②・for 文の書き方 ③ p.11〜13

G **組み合わせの型**
型① 全部に同じこと ／ 型② 条件に合うものだけ・型③ 数える ／ 型④ 合計する・型⑤ 最大・最小を見つける など p.14〜19

H **文字列**
文字列の基本・文字列メソッド 一覧・切り取る・探す・書き換える など p.20〜24

I **よくあるエラー**
よくあるエラーと直し方 p.25

A

基本の形

Paiza の書き始め（毎回この形）

C# Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
class Program
{
    static void Main()
    {
        // ここに処理を書く
    }
}
```

書くもの	使えるようになるもの
using System;	Console・int.Parse など
using System.Collections.Generic;	List など
using System.Linq;	Select・ToArray など

- using の 3 行は**常に書く**
- 処理は Main の { } の中に書く
- 関数を作るときは Main の外、class Program の中に static で書く

POINT 迷ったら、まずこの形をコピーしてから書き始めよう。

B

入力

1 行を受け取る

1 行の文字列

そのまま string で受け取る。ReadLine は 1 回で 1 行読む。

```
string text = Console.ReadLine();
```

1 行に数値 1 つ

文字列で受け取ってから int.Parse で整数に。

```
string text = Console.ReadLine();
int number = int.Parse(text);
```

1 行に小数 1 つ

小数は double.Parse を使う。

```
string text = Console.ReadLine();
double value = double.Parse(text);
```

1 行に 1 文字

受け取った文字列の [0] で char にする。

```
string text = Console.ReadLine();
char c = text[0];
```

B

入力

スペース区切りを受け取る

スペース区切りの文字

Split(" ") で切り分けて string 配列に。

```
string text = Console.ReadLine();
string[] words = text.Split(" ");
```

スペース区切りの数値

Split → Select(int.Parse) → ToArray で int 配列に。（using System.Linq; が必要）

```
string text = Console.ReadLine();
int[] numbers = text.Split(" ").Select(int.Parse).ToArray();
```

2 つの数値を別々の変数に

例：「N M」の 1 行。配列で受け取ってから取り出す。

```
string text = Console.ReadLine();
int[] nm = text.Split(" ").Select(int.Parse).ToArray();
int n = nm[0];
int m = nm[1];
```

文字と数値がまざった行

例：「swap 3 5」。文字列で切ってから、数値の部分だけ int.Parse。

```
string[] words = Console.ReadLine().Split(" ");
string command = words[0];
int a = int.Parse(words[1]);
```

B

入力

複数行を受け取る

N 行に数値が 1 つずつ

1 行目の N の回数だけ for で ReadLine。配列に入れておく。

```
int n = int.Parse(Console.ReadLine());
int[] values = new int[n];
for (int i = 0; i < n; i++)
{
    values[i] = int.Parse(Console.ReadLine());
}
```

N 行に「名前 点数」

for の中で 1 行ずつ Split して取り出す。

```
int n = int.Parse(Console.ReadLine());
for (int i = 0; i < n; i++)
{
    string[] words = Console.ReadLine().Split(" ");
    string name = words[0];
    int score = int.Parse(words[1]);
}
```

1 行目 N、2 行目に N 個の数値

1 行目は数値 1 つ、2 行目はスペース区切りの数値。

```
int n = int.Parse(Console.ReadLine());
int[] numbers = Console.ReadLine().Split(" ").Select(int.Parse).ToArray();
```

C

出力

出力する

改行あり / なし

WriteLine は最後に改行する。Write は改行しない。

```
Console.WriteLine("Hello");
Console.Write("A");
Console.Write("B");
Console.WriteLine();
```

文字と変数をつなげて出力

+ でつなげる。計算は ( ) で先にする。

```
Console.WriteLine(name + " " + score);
Console.WriteLine("合計:" + (a + b));
```

配列をスペース区切りで 1 行に

string.Join(区切り, 配列)。最後に余計な空白がつかない。

```
Console.WriteLine(string.Join(" ", numbers));
```

小数点以下の桁をそろえる

ToString("F2") で小数第 2 位まで（3.14159 → 3.14）。

```
Console.WriteLine(value.ToString("F2"));
```

D

計算

計算でよく使う書き方

余り %：偶数・奇数・倍数

割った余りが 0 なら割り切れる。

```
if (n % 2 == 0) { /* 偶数 */ }
if (n % 3 == 0) { /* 3 の倍数 */ }
```

割り算の切り捨てと小数

int どうしの / は切り捨て。小数にしたいときは double にする。

```
Console.WriteLine(7 / 2);
Console.WriteLine(7 / 2.0);
Console.WriteLine((double)a / b);
```

大きい方・小さい方・絶対値

Math を使う。

```
int bigger = Math.Max(a, b);
int smaller = Math.Min(a, b);
int distance = Math.Abs(a - b);
```

増やす・減らすの省略形

total += x は total = total + x と同じ。

```
total += x;
hp -= damage;
count++;
count--;
```

E

if 文

if 文の書き方

もし〜なら / そうでなければ

( ) が true なら if の中、false なら else の中を実行。

```
if (hp ⩽ 0)
{
    Console.WriteLine("Game Over");
}
else
{
    Console.WriteLine("Continue");
}
```

3 つ以上に分ける（else if）

上から順に調べて、最初に true の所だけ実行。**きびしい条件を先に**書く。

```
if (score ⩾ 80)
{
    Console.WriteLine("A");
}
else if (score ⩾ 60)
{
    Console.WriteLine("B");
}
else
{
    Console.WriteLine("C");
}
```

E

if 文

条件の書き方（比較・かつ・または）

書き方	意味	例
a == b	等しい	hp == 0
a != b	等しくない	hp != 0
a < b ／ a > b	より小さい ／ より大きい	score > 80
a <= b ／ a >= b	以下 ／ 以上	score >= 60
条件 && 条件	かつ（両方 true）	score >= 60 && score < 80
条件 || 条件	または（どちらか true）	a == 0 || b == 0
!条件	〜ではない	!char.IsDigit(c)
text == "yes"	文字列が等しい	answer == "yes"
c == 'a'	文字（char）が等しい	text[0] == 'a'

注意　「60 以上 80 未満」は 60 <= score < 80 とは書けない。score >= 60 && score < 80 と 2 つに分ける。

F

for 文

for 文の書き方 ①

N 回くり返す（基本の形）

i は 0, 1, …, n-1。迷ったらこれ。

```
for (int i = 0; i < n; i++)
{
    // N 回実行される
}
```

1 から N まで

i の値そのものを使いたいとき。

```
for (int i = 1; i ≤ n; i++)
{
    Console.WriteLine(i);
}
```

配列を全部見る

回数は 配列名.Length。

```
for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine(numbers[i]);
}
```

F

for 文

for 文の書き方 ②

後ろから順に見る

Length - 1 から 0 まで、i-- で減らす。

```
for (int i = numbers.Length - 1; i ⩾ 0; i--)
{
    Console.WriteLine(numbers[i]);
}
```

foreach（番号がいらないとき）

配列・文字列の中身を先頭から 1 つずつ取り出す。

```
foreach (int x in numbers)
{
    Console.WriteLine(x);
}
```

break / continue

break：for をすぐ抜ける。continue：今の周を飛ばして次の周へ。

```
for (int i = 0; i < n; i++)
{
    if (numbers[i] < 0) { continue; }
    if (numbers[i] == 0) { break; }
}
```

F

for 文

for 文の書き方 ③（2 重の for）

表を作る（for の中に for）

外側が「行」、内側が「列」。内側で 1 行分を作って、外側で出力する。Trim() で最後の空白を消す。

```
for (int y = 1; y ≦ 3; y++)
{
    string line = "";
    for (int x = 1; x ≦ 3; x++)
    {
        line += (x * y) + " ";
    }
    Console.WriteLine(line.Trim());
}
```

出力

```
1 2 3
2 4 6
3 6 9
```

POINT

内側の for は、外側の 1 周ごとに最初から全部回る。外側 3 回 × 内側 3 回 ＝ 中身は 9 回実行される。

G

組み合わせの型

型① 全部に同じこと ／ 型② 条件に合うものだけ

型① 全部に同じことをする

ことば：「それぞれ」「すべて」
for だけ。if はいらない。

```
for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine(numbers[i] * 2);
}
```

型② 条件に合うものだけ

ことば：「〜なら」「〜だけ」
for の中に if を入れる。

```
for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] % 2 == 0)
    {
        Console.WriteLine(numbers[i]);
    }
}
```

G

組み合わせの型

型③ 数える ／ 型④ 合計する

型③ 数える

ことば：「何個」「何人」「何回」

前：count = 0

中：条件に合えば count++

後：出力

```
int count = 0;
for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] ⩾ 60)
    {
        count++;
    }
}
Console.WriteLine(count);
```

型④ 合計する

ことば：「合計」「全部で」

前：total = 0

中：total += 値

後：出力

```
int total = 0;
for (int i = 0; i < numbers.Length; i++)
{
    total += numbers[i];
}
Console.WriteLine(total);
```

G

組み合わせの型

型⑤ 最大・最小を見つける

型⑤ 最大

ことば：「一番大きい」「最高」
前：max = 最初の値
中：今の max より大きければ入れかえ
後：出力

```
int max = numbers[0];
for (int i = 1; i < numbers.Length; i++)
{
    if (numbers[i] > max)
    {
        max = numbers[i];
    }
}
Console.WriteLine(max);
```

型⑤ 最小

ことば：「一番小さい」「最低」
最大の > を < にするだけ。
Math.Max / Math.Min を使ってもよい。

```
int min = numbers[0];
for (int i = 1; i < numbers.Length; i++)
{
    min = Math.Min(min, numbers[i]);
}
Console.WriteLine(min);
```

G

組み合わせの型

型⑥ あるかないか（フラグ）

型⑥ あるか

ことば：「含まれているか」「Yes / No」
前：isFound = false
中：見つけたら true
後：true / false で Yes / No
「すべて〜か」は true から始めて、ちがうものがあれば false。

```
bool isFound = false;
for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] == target)
    {
        isFound = true;
    }
}
if (isFound)
{
    Console.WriteLine("Yes");
}
else
{
    Console.WriteLine("No");
}
```

注意 Yes / No の出力は for の**後**。for の中に else で "No" を書くと、何回も No が出てしまう。

組み合わせの型

型⑦ 見つけたら止める ／ 型の組み合わせ

型⑦ 見つけたら止める

ことば：「最初の」「何番目」
前：position = -1（見つからない用）
中：見つけたら記録して break
「何番目」は i + 1

```
int position = -1;
for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] == target)
    {
        position = i + 1;
        break;
    }
}
Console.WriteLine(position);
```

組み合わせ：条件つきの数と合計

1 つの if の中に、型③と型④を並べて書く。
条件は && でつなげてもよい。

```
int count = 0;
int total = 0;
for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] ⩾ 60)
    {
        count++;
        total += numbers[i];
    }
}
```

G

組み合わせの型

型の一覧（for の前・中・後）

型	問題文のことば	① for の前	② for の中	③ for の後
① 全部に同じこと	それぞれ・すべて	—	処理する	—
② 条件に合うものだけ	～なら・～だけ	—	if (条件) { 処理 }	—
③ 数える	何個・何人	count = 0	if (条件) count++	count を出力
④ 合計する	合計・全部で	total = 0	total += 値	total を出力
⑤ 最大・最小	一番・最高・最小	max = 最初の値	大きければ入れかえ	max を出力
⑥ あるかないか	含まれているか	isFound = false	見つけたら true	Yes / No
⑦ 見つけたら止める	最初の・何番目	position = -1	記録して break	position を出力

POINT 迷ったら「この行は何回実行したい？」。1 回 → for の前か後 ／ N 回 → for の中。

H

文字列

文字列の基本

文字数と i 文字目

番号は 0 から。問題文の「i 文字目」は text[i - 1]。取り出すと char になる。

```
int length = text.Length;
char first = text[0];
char last = text[text.Length - 1];
char ith = text[i - 1];
```

1 文字ずつ見る

配列と同じ。番号がいらなければ foreach。

```
for (int i = 0; i < text.Length; i++)
{
    Console.WriteLine(text[i]);
}
foreach (char c in text)
{
    Console.WriteLine(c);
}
```

文字列を組み立てる

空の文字列 "" から += でつなげる。

```
string result = "";
result += "A";
result += c;
```

H

文字列

文字列メソッド 一覧

書き方	意味	例（text = "paiza"）→ 結果
text.Length	文字数	5
text.ToUpper() / ToLower()	大文字 / 小文字にする	"PAIZA"
text.Substring(開始, 文字数)	一部を切り取る	Substring(1, 3) → "aiz"
text.Contains(文字列)	含まれているか	Contains("iz") → True
text.IndexOf(文字)	何番目にあるか（なければ -1）	IndexOf('z') → 3
text.Replace(前, 後)	全部置きかえる	Replace("a", "A") → "pAizA"
text.Split(" ")	区切って配列にする	"a b" → { "a", "b" }
string.Join(" ", 配列)	配列をつないで 1 つの文字列に	{ "a", "b" } → "a b"
text.Trim()	前後の空白を消す	" ab " → "ab"
new string('*', 5)	同じ文字を並べる	"*****"

H

文字列

切り取る・探す・書き換える

i 文字目から j 文字目まで

Substring の 2 つ目は「終わりの番号」ではなく「文字数」。

```
string part = text.Substring(i - 1, j - i + 1);
```

何文字目にあるか / 含まれているか

IndexOf は 0 から数えるので +1。見つからないと -1。

```
int position = text.IndexOf(c) + 1;
if (text.Contains("abc"))
{
    Console.WriteLine("YES");
}
```

i 文字目だけ書き換える

文字列は直接書き換えられない（text[0] = 'P' はエラー）。char 配列にしてから。

```
char[] chars = text.ToCharArray();
chars[i - 1] = 'X';
string result = new string(chars);
```

H

文字列

1 文字の判定と、型の変換

書き方	意味
char.IsUpper(c)	大文字？
char.IsLower(c)	小文字？
char.IsDigit(c)	数字？
char.IsLetter(c)	アルファベットなどの文字？
char.ToUpper(c)	大文字にする
char.ToLower(c)	小文字にする

変換	書き方
string → int	int.Parse(text)
int → string	number.ToString()
string → char	text[0]
char → string	c.ToString()
数字の char → int	c - '0'
string → char 配列	text.ToCharArray()
char 配列 → string	new string(chars)

注意 char は ' '、string は " "。text[0] == "a" はエラーになる（text[0] == 'a' が正しい）。

H

文字列

文字列でよく使う型

ある文字を数える（型③）

foreach で 1 文字ずつ見て、一致したら count++。

```
int count = 0;
foreach (char c in text)
{
    if (c == 'a') { count++; }
}
```

反転する（型④の文字列版）

後ろからの for で 1 文字ずつつなげる。

```
string reversed = "";
for (int i = text.Length - 1; i >= 0; i--)
{
    reversed += text[i];
}
```

すべて数字か（型⑥）

「すべて〜か」は true から始める。

```
bool isAllDigit = true;
foreach (char c in text)
{
    if (!char.IsDigit(c)) { isAllDigit = false; }
}
```

I

よくあるエラー

よくあるエラーと直し方

こんなとき	よくある原因	直し方
IndexOutOfRangeException（範囲外）	i <= Length で回した／text[i] の i が 1 ずれている	i < Length にする／「i 文字目」は [i - 1]
FormatException（形式が違う）	スペースの入った行や文字を int.Parse した	Split してから 1 つずつ int.Parse
char と string は比べられない	text[0] == "a" と書いた	text[0] == 'a'
名前が存在しない	for や if の中で作った変数を、外で使った	変数は for の前で作る
出力が多すぎる	WriteLine を for の中に書いた	結果の出力は for の後へ
答えがずれる・合わない	= と == の間違い／else if の順番	比較は ==／きびしい条件を先に

注意 エラーが出たら、メッセージの「行番号」を見て、その行と 1 つ前の行を確認しよう。